

- welcome to
claude code

boris cherny

tl;dr

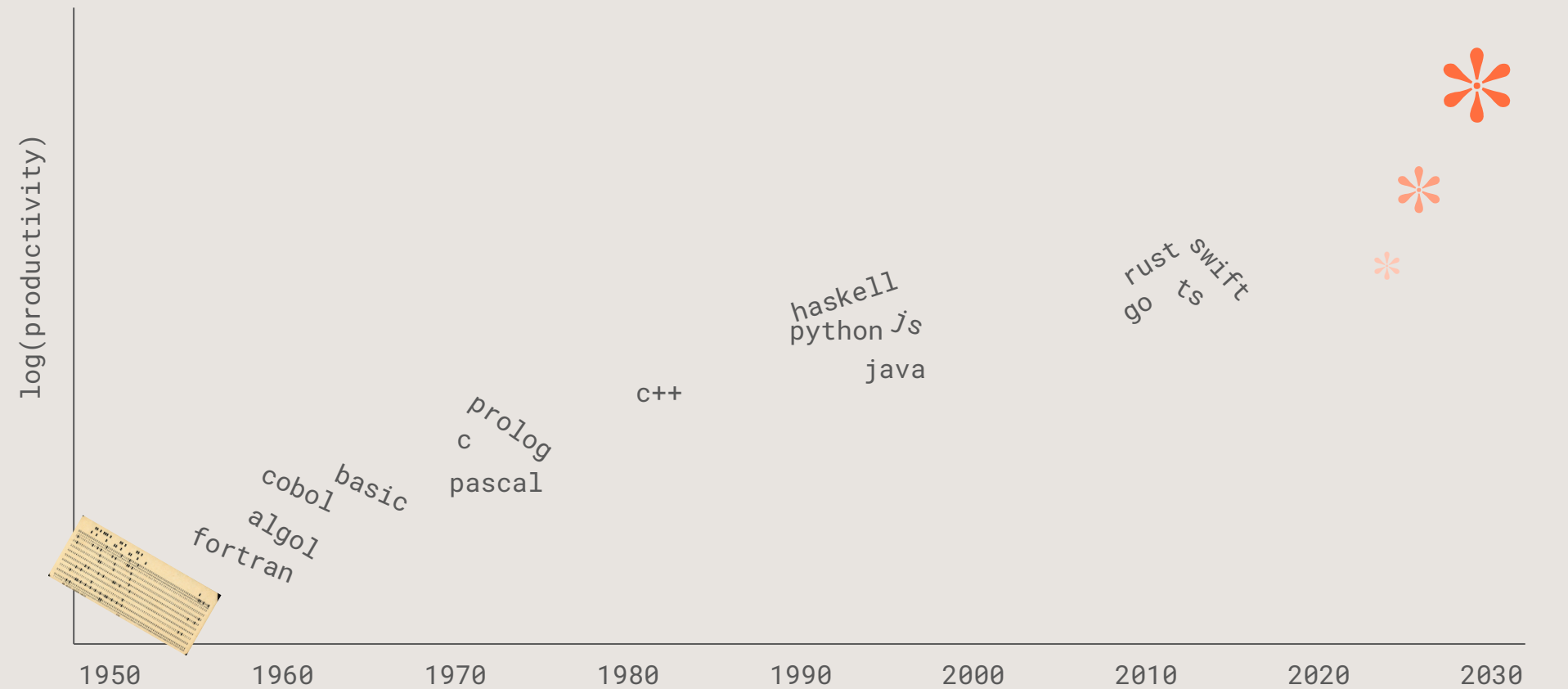
1. programming is changing
2. choose your path with claude code
3. think six months out

```
npm install -g @anthropic-ai/claude-code
```

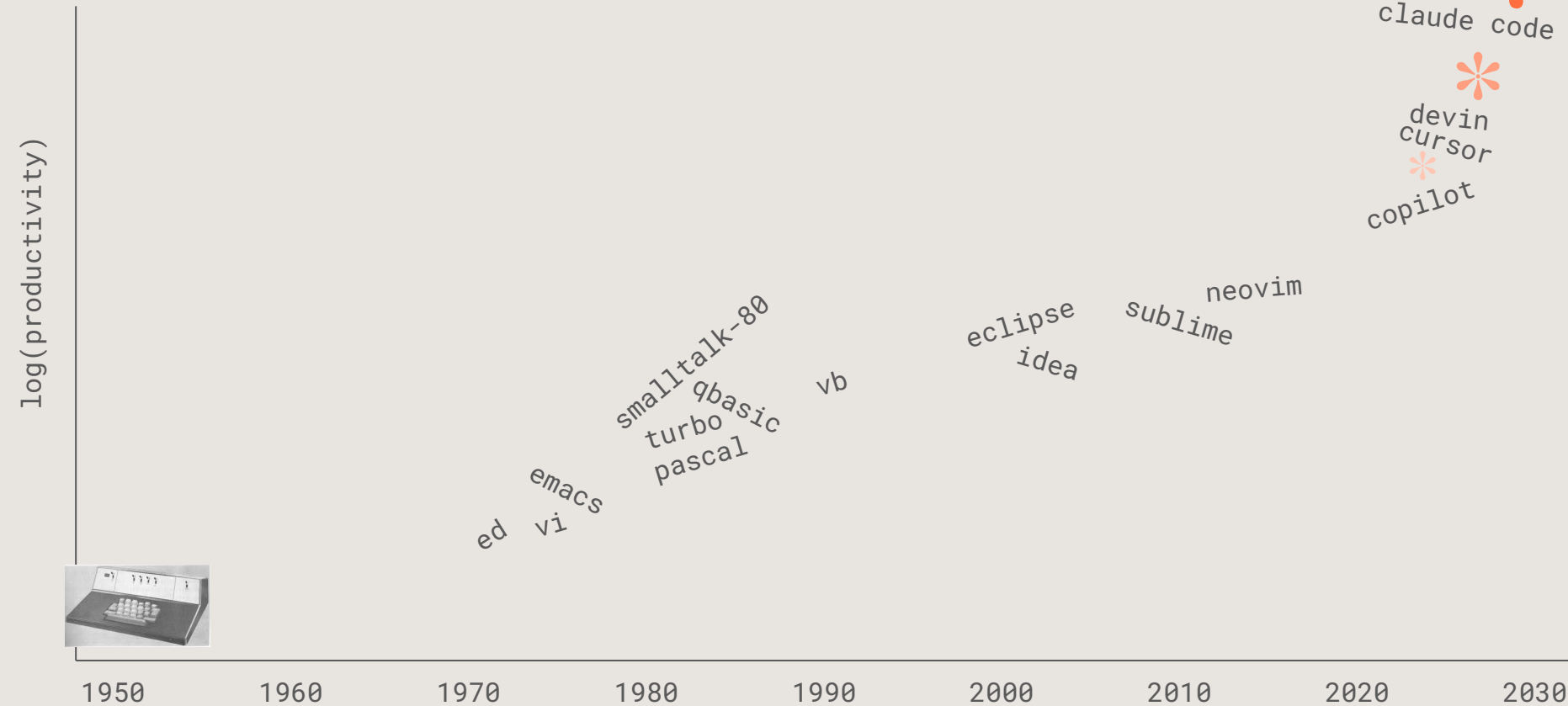
```
claude.ai/code
```

1/ programming is at an inflection point

programming language productivity is
increasing exponentially, driven by ai



ide productivity is following a similar
exponential, also driven by ai

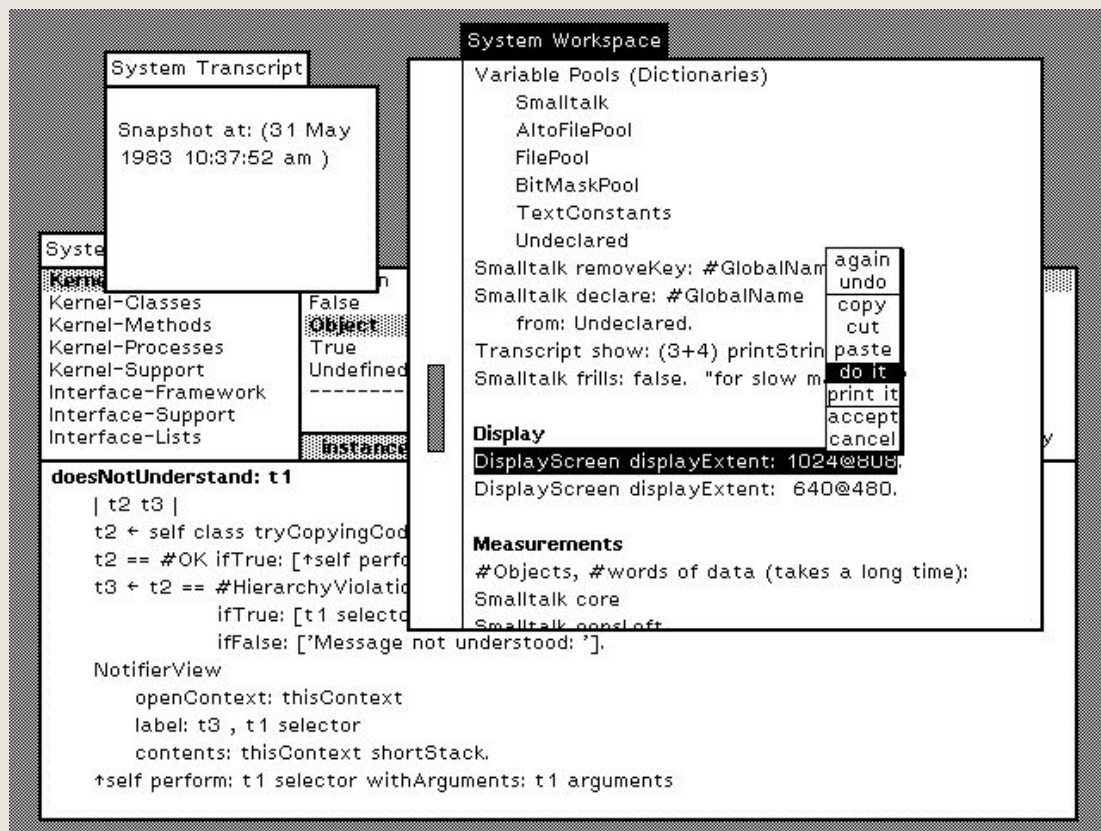


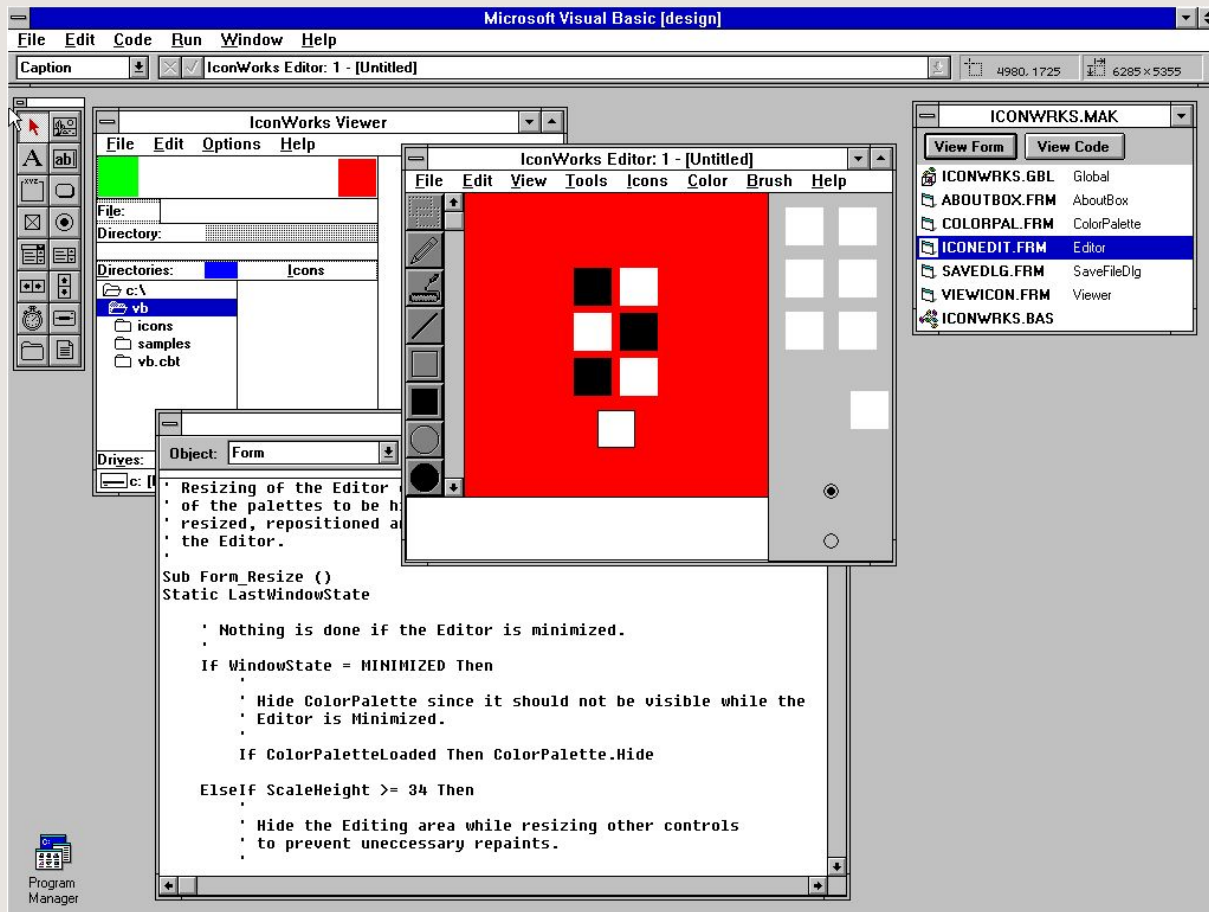


ibm 029 (1964)

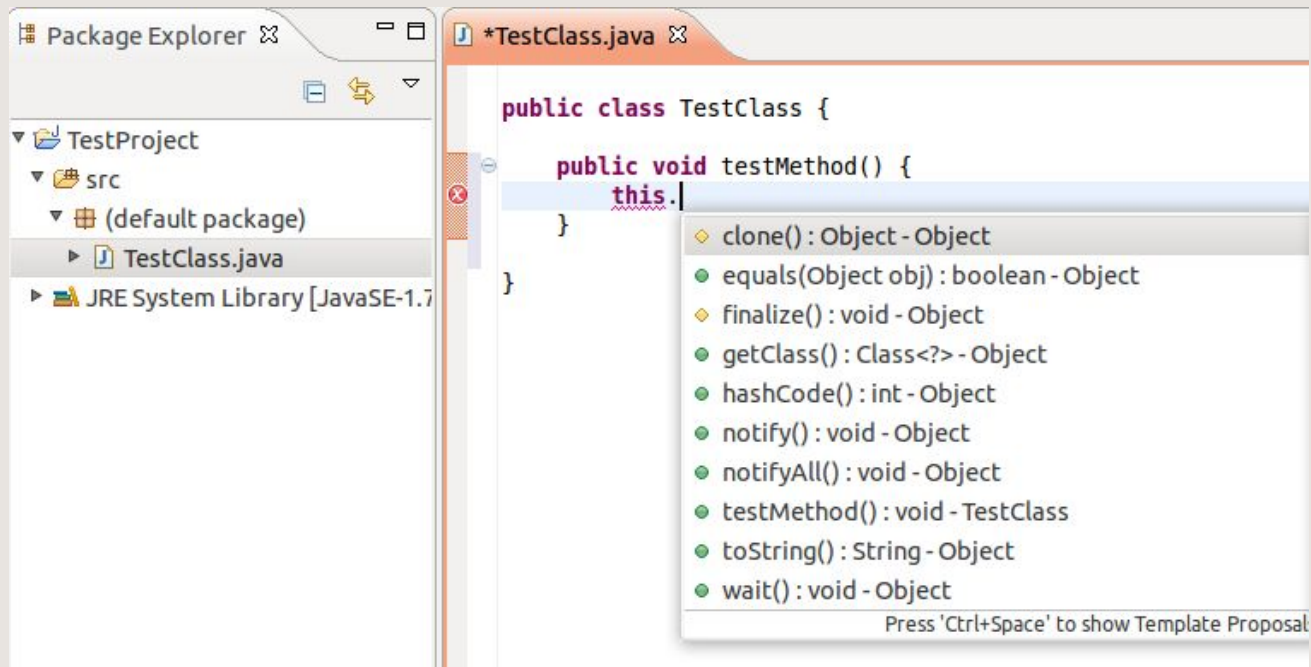
```
$ ed fstab
Newline appended
116
%1
/dev/hda2 / ext2 defaults 1 1\r$
/dev/hdb1 /home ext2 defaults 1 2\r$
/dev/hda1 swap swap pri=40 0 0$
3s/40/42/
w fstab
116
—
```

ed (1969)





visual basic (1991)



eclipse (2001)

```

1  from __future__ import print_function
2  import argparse
3  import torch
4  import torch.nn as nn
5  import torch.optim as optim
6  import numpy as np
7  import matplotlib
8  matplotlib.use('Agg')
9  import matplotlib.pyplot as plt
10
11 class Sequence(nn.Module):
    def __init__(self):
        super(Sequence, self).__init__()
        self.lstm1 = nn.LSTMCell(1, 51)
        self.lstm2 = nn.LSTMCell(51, 51)
        self.linear = nn.Linear(51, 1)

    def forward(self, input, future = 0):
        outputs = []
        h_t = torch.zeros(input.size(0), 51, dtype=torch.double)

```



Top 10 Programming Competitions



**You** May 29, 2024, 2:02 AM

Create a website with a list of the top 10 programming competitions in the world and make sure that each one on the list links to the competition's main website

**Devin** May 29, 2024, 2:02 AM

Got it! I'll create a website listing the top 10 programming competitions in the world, each linking to their main website. I'll get started on this right away.

**Devin** May 29, 2024, 2:03 AM

I'm going to set up a React app to create the website listing the top 10 programming competitions, each linking to their main website. I'll use Chakra UI for a modern and user-friendly design.

Quick question: Are there any specific programming competitions you want to be included in the list, or should I choose them independently? Also, do you have any preferences for the design and layout of the website (e.g., color scheme, fonts, etc.)?

If I don't hear back, I'll go ahead and choose the competitions and use a clean, modern design by default.



Set up a new React app using Create React App.



Installed Chakra UI and configured it for the project.



Created a component to list the top 10 programming competitions with links to their main websites.



Styled the component to ensure a modern and user-friendly design.



Ran the local development server and verified the website functionality.



Built the React app for production.



Session Ended.





Start a new conversation with Devin

If you'd like to evaluate this run, [click here](#).

Devin's Workspace

Following  

Shell Browser Editor  Planner

App.js



home > ubuntu > top10comps > src > App.js

```
1 import React from 'react';
2 import { ChakraProvider, Box, Link, Text, VStack, Heading } from '@chakra-ui/react';
3
4 function App() {
5   const competitions = [
6     { name: 'ACM ICPC', url: 'https://icpc.global/' },
7     { name: 'Google Code Jam', url: 'https://codingcompetitions.withgoogle.com/codejam' },
8     { name: 'Facebook Hacker Cup', url: 'https://www.facebook.com/codingcompetitions/hacker-cup' },
9     { name: 'Codeforces', url: 'https://codeforces.com/' },
10    { name: 'TopCoder', url: 'https://www.topcoder.com/' },
11    { name: 'AtCoder', url: 'https://atcoder.jp/' },
12    { name: 'LeetCode', url: 'https://leetcode.com/contest/' },
13    { name: 'HackerRank', url: 'https://www.hackerrank.com/domains/tutorials/10-days-of-javascript' },
14    { name: 'Kick Start', url: 'https://codingcompetitions.withgoogle.com/kickstart' },
15    { name: 'CodeChef', url: 'https://www.codechef.com/' },
16  ];
17
18  return (
19    <ChakraProvider>
20      <VStack spacing={8} align="stretch" p={5}>
21        <Heading as="h1" size="2xl" mb={5} textAlign="center">
22          Top 10 Programming Competitions
23        </Heading>
24        <Box key={index} p={5} shadow="md" borderWidth="1px" borderRadius="md" data-cs={2} data-kind="parent" data-bbox="580 624 868 768" data-expected-kind="parent" data-rs="2">
25          <Link href={comp.url} isExternal>
26            <Text fontSize="xl" fontWeight="bold" color="teal.500">
27              {comp.name}
28            </Text>
29          </Link>
30        </Box>
31      </VStack>
32    </ChakraProvider>
33  );
34 }
```



Create machine snapshot 

devin (2024)

ide devx has evolved quickly, and will
continue to change even more quickly

verification is evolving quickly, too

- manual debugging
- static types (algol)
- formal verification
- abstract interpretation
- automated testing
- continuous integration
- property-based testing (quickcheck)
- dependent typing
- e2e testing
- chaos testing (chaos monkey)
- ai-powered vulnerability testing
- ai-powered unit testing (testgen)
- ai-powered fuzz testing (sapienz)
- self play
- ...

claude code's approach:
works everywhere

1. terminal-native
2. low-level model access
3. infinitely hackable

works across the whole sdlc

1. Discover

Explore codebase
and history

Search
documentation

Onboard & learn

2. Design

Plan project

Develop tech specs

Define architecture

3. Build

Implement code

Write and execute
tests

Create commits
and PRs

4. Deploy

Automate CI/CD

Configure
environments

Manage
deployments

5. Support & Scale

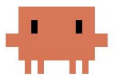
Debug errors

Large-scale
refactor

Monitor usage &
performance

Using and mastering all of your team's CLI tools (e.g., git, docker, bq) so you can focus on solutions, not syntax

ⓧ Invalid API Keys 🔍 %1

 **Claude Code** v2.0.32
Sonnet 4.5 · API Usage Billing
/Users/boris/code/claude-cli

> brainstorm then implement a better solution for when api keys are invalid
└ Async hook **SessionStart** completed

- I'll help you brainstorm and implement a better solution for handling invalid API keys. Let me first explore the current implementation to understand how API key validation is handled.

Explore(Explore API key validation)
└ **Search**(pattern: "401|unauthorized")
 Search(pattern: "validateApiKey|validate.*key")
 +1 more tool use (ctrl+o to expand)

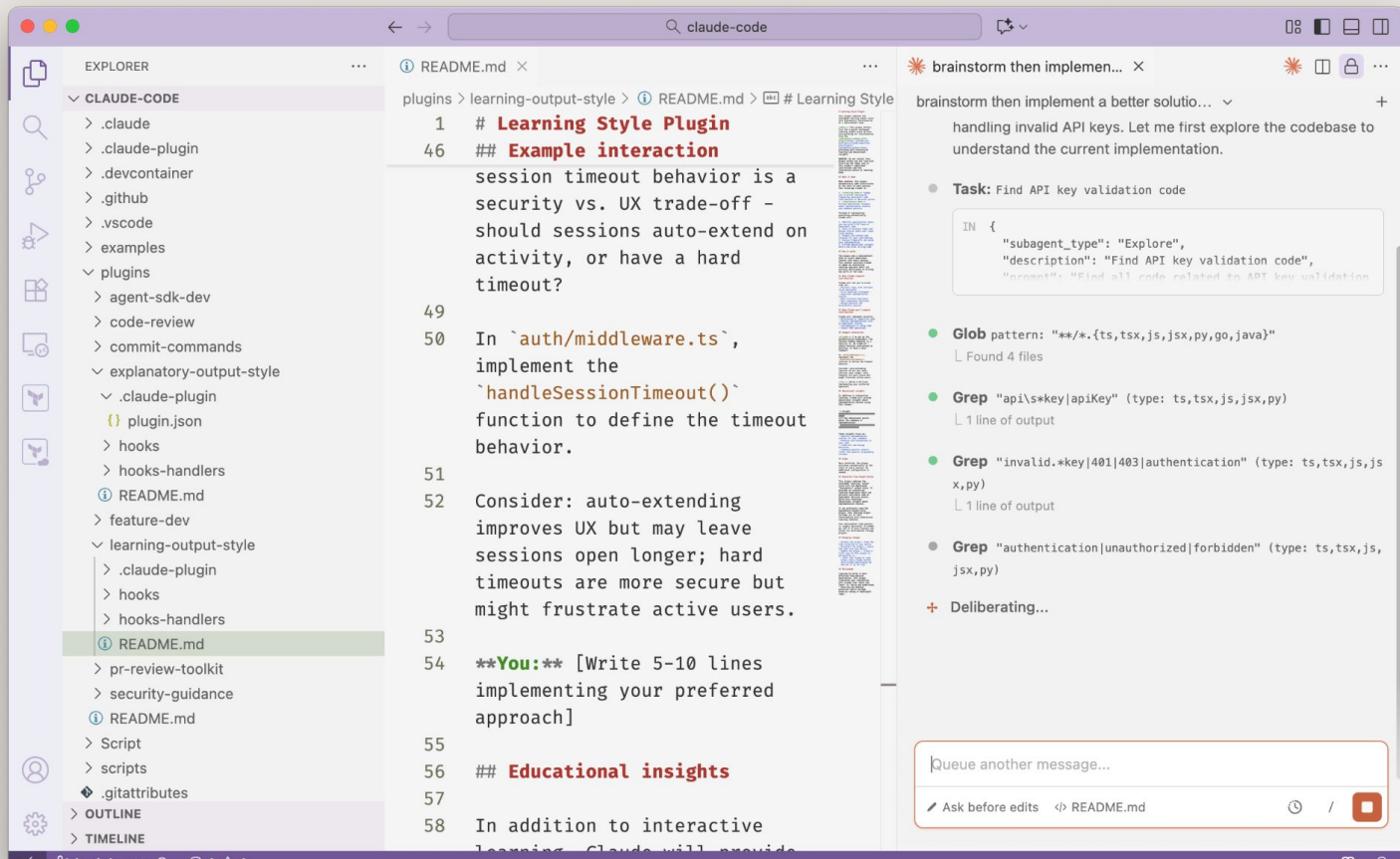
* Moseying... (esc to interrupt)

> █

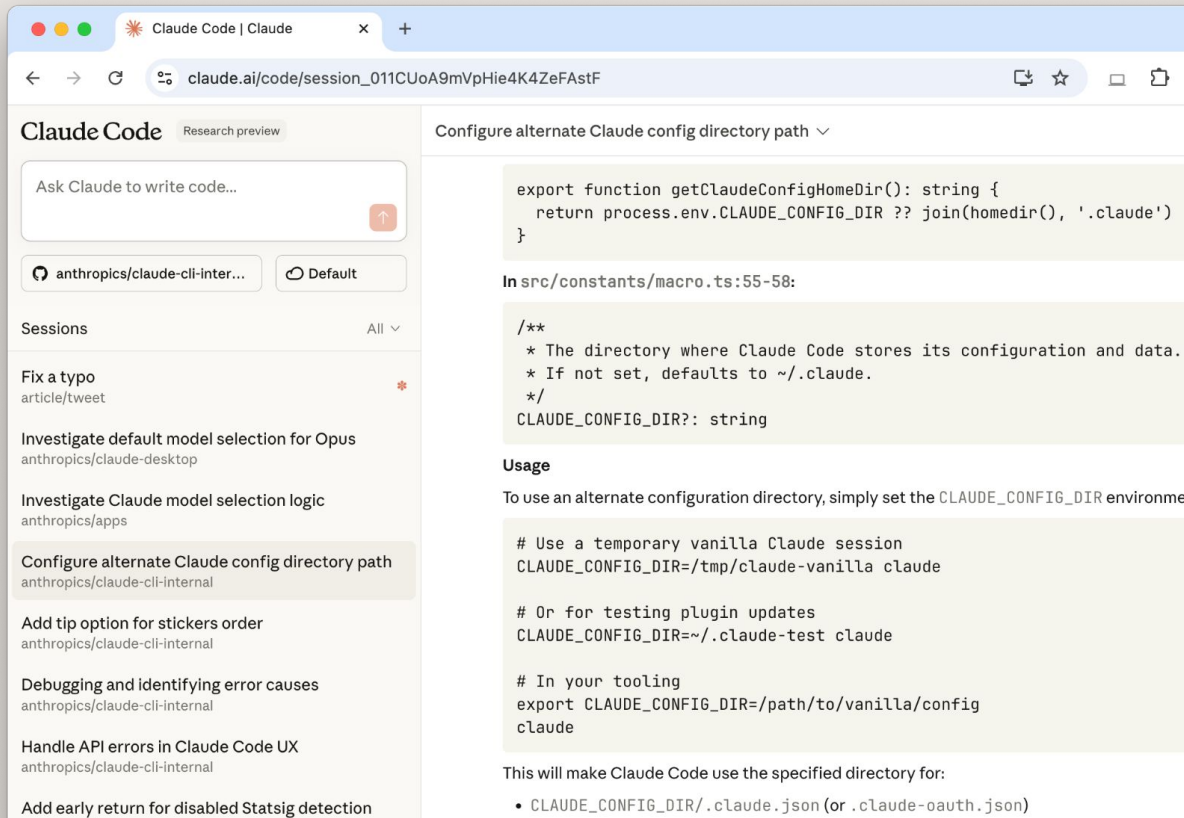
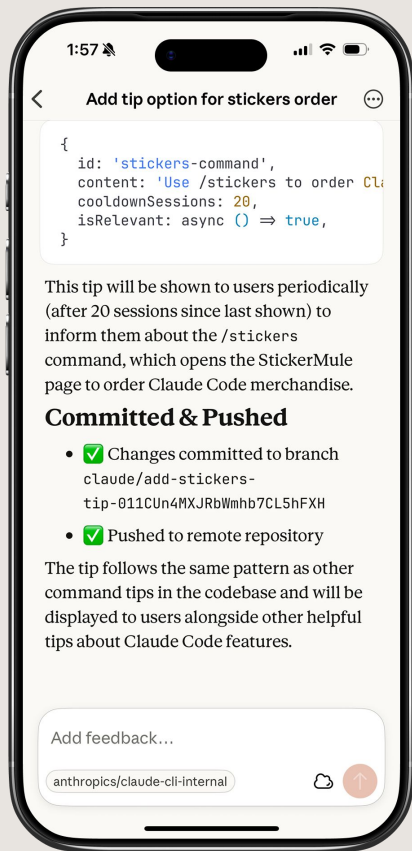
? for shortcuts Thinking off (tab to toggle)

terminal

ide



web & ios



/install-github-app

The screenshot shows a web browser window displaying a GitHub issue page. The browser's address bar shows the URL `github.com/bcherny/json-schema-to-typescript/issues/660`. The issue title is "Parsed ts doesn't have description when string array with description #660".

The issue is owned by **bcherny** and was posted 1 minute ago. The content of the issue is:

@claude implement this

claude responded 1 minute ago, edited by claude:

Working on implementing string array item descriptions 🌟

I'll implement the fix for issue [#660](#) where string array item descriptions are missing from generated TypeScript.

Todo List:

- ☒ Read repository setup instructions (CLAUDE.md) - No CLAUDE.md found, checked README and package.json
- ☒ Understand the codebase structure and current description handling
- ☒ Find array processing logic and identify where item descriptions should be added
- ☐ Implement the fix to include array item descriptions
- ☐ Test with the provided example
- ☐ Run linting/testing commands (need Bash permissions)

sdk

```
$ claude -p \  
  "what did i do this week?" \  
  --allowedTools Bash(git log:*)  
  --output-format stream-json
```

your app

claude code sdk

anthropic, bedrock, or vertex api

claude models

sdk

```
$ get-gcp-logs 1uhd832d |  
  claude -p "correlate errors + commits" \  
    --output-format=json |  
jq '.result'
```

your app

claude code sdk

anthropic, bedrock, or vertex api

claude models

use cases

1. codebase q&a + research
2. write code
 - a. 1-shot
 - b. sidekick
 - c. prototype
3. integrate tools & mcps
4. power automation

1. ask claude code about your code

> how do I make a new `@app/services/ValidationTemplateFactory`?

> why does `recoverFromException` take so many arguments? look through git history to answer

> why did we fix issue #18363 by adding the if/else in `@src/login.ts` api?

> in which version did we release the new `@api/ext/PreHooks.php` api?

> look at PR #9383, then carefully verify which app versions were impacted

> what did I ship last week?

2. teach claude to use your tools

> use the barley cli to check for error logs

```
$ claude mcp add barley_server -- node myserver
```

> use the barley mcp server to check for error logs

3. fit the workflow to the task

explore › plan › confirm › code › commit

> figure out the root cause for issue #983, then propose a few fixes. Let me choose an approach before you code. ultrathink

3. fit the workflow to the task

tests › commit › code › iterate › commit

> write tests for `@utils/markdown.ts` to make sure links render properly

(note the tests won't pass yet, since links aren't yet implemented). then

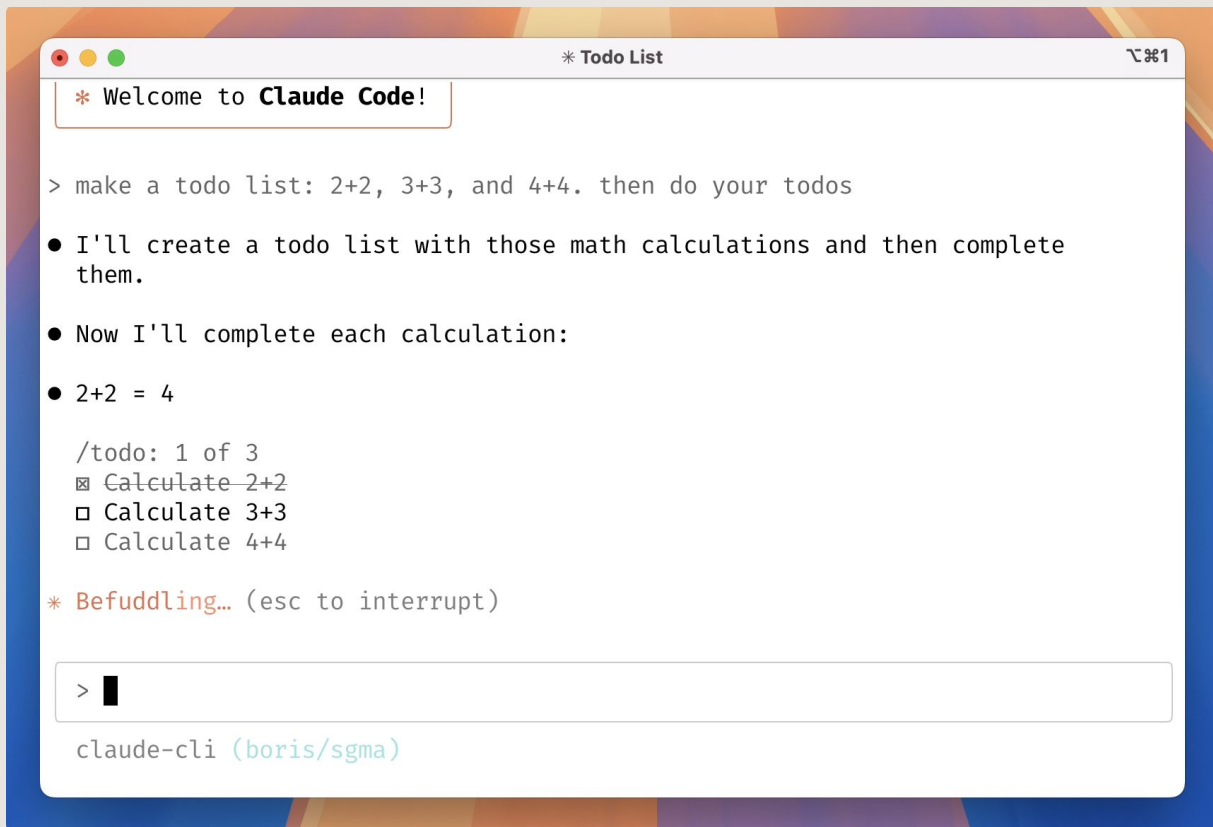
commit. then update the code to make the tests pass.

3. fit the workflow to the task

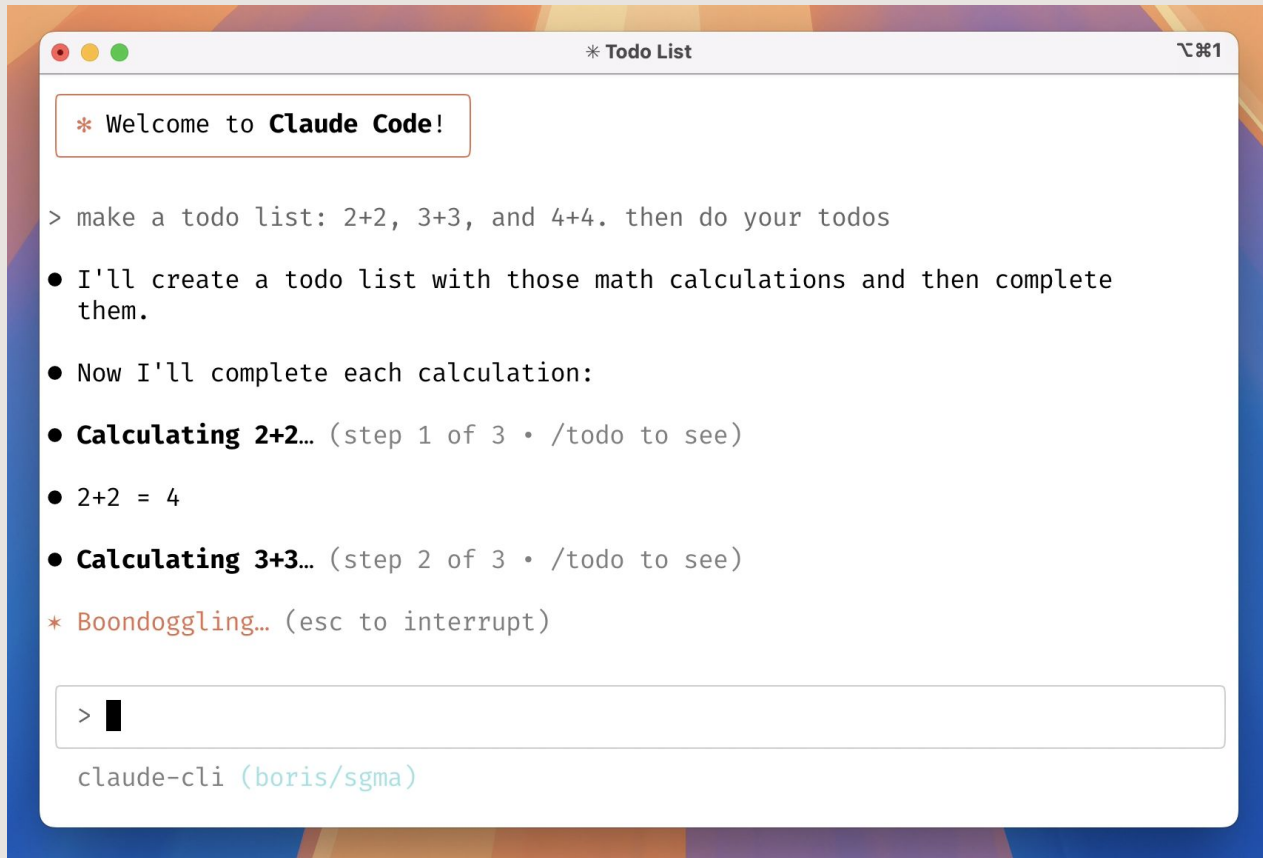
code › screenshot › iterate

> implement [mock.png]. Then screenshot it with puppeteer and iterate till it looks like the mock.

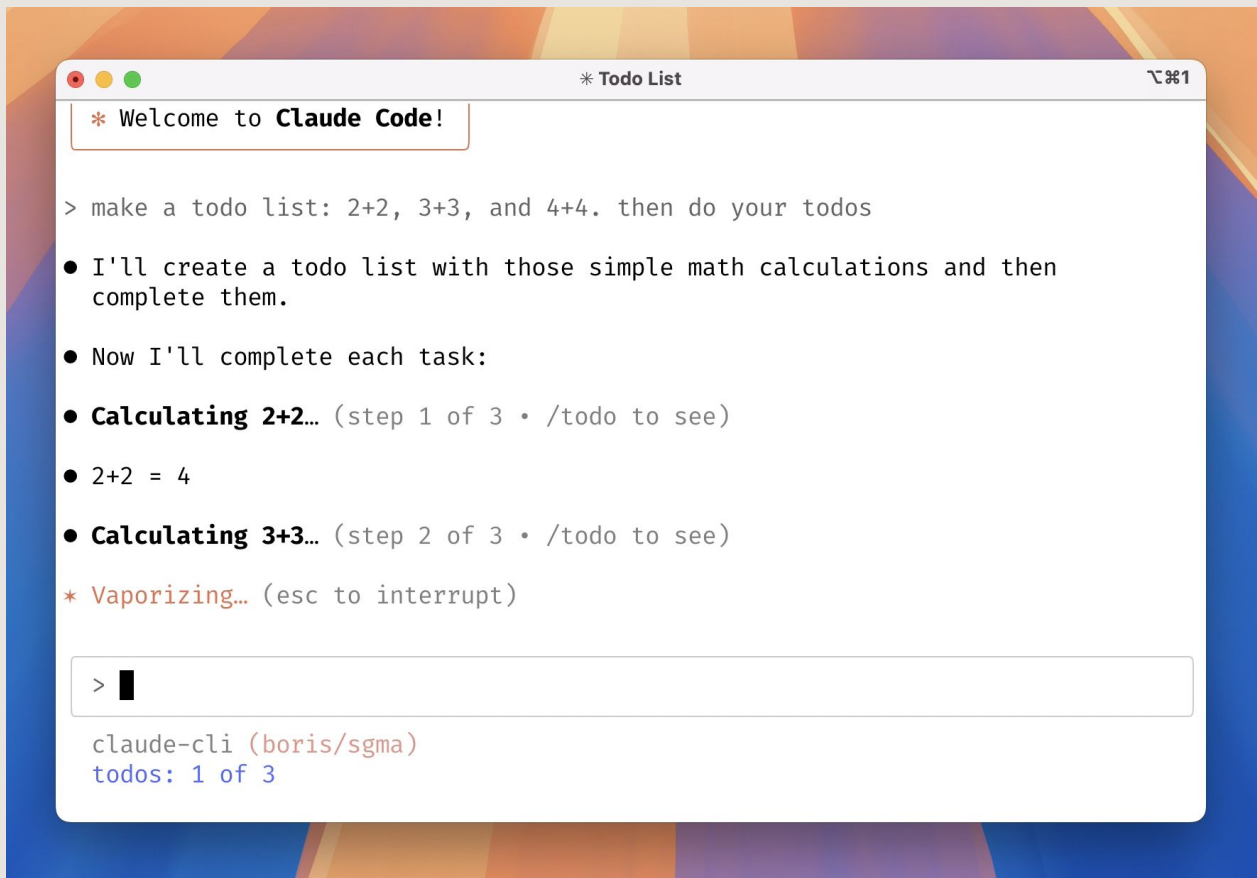
4. use claude code to prototype



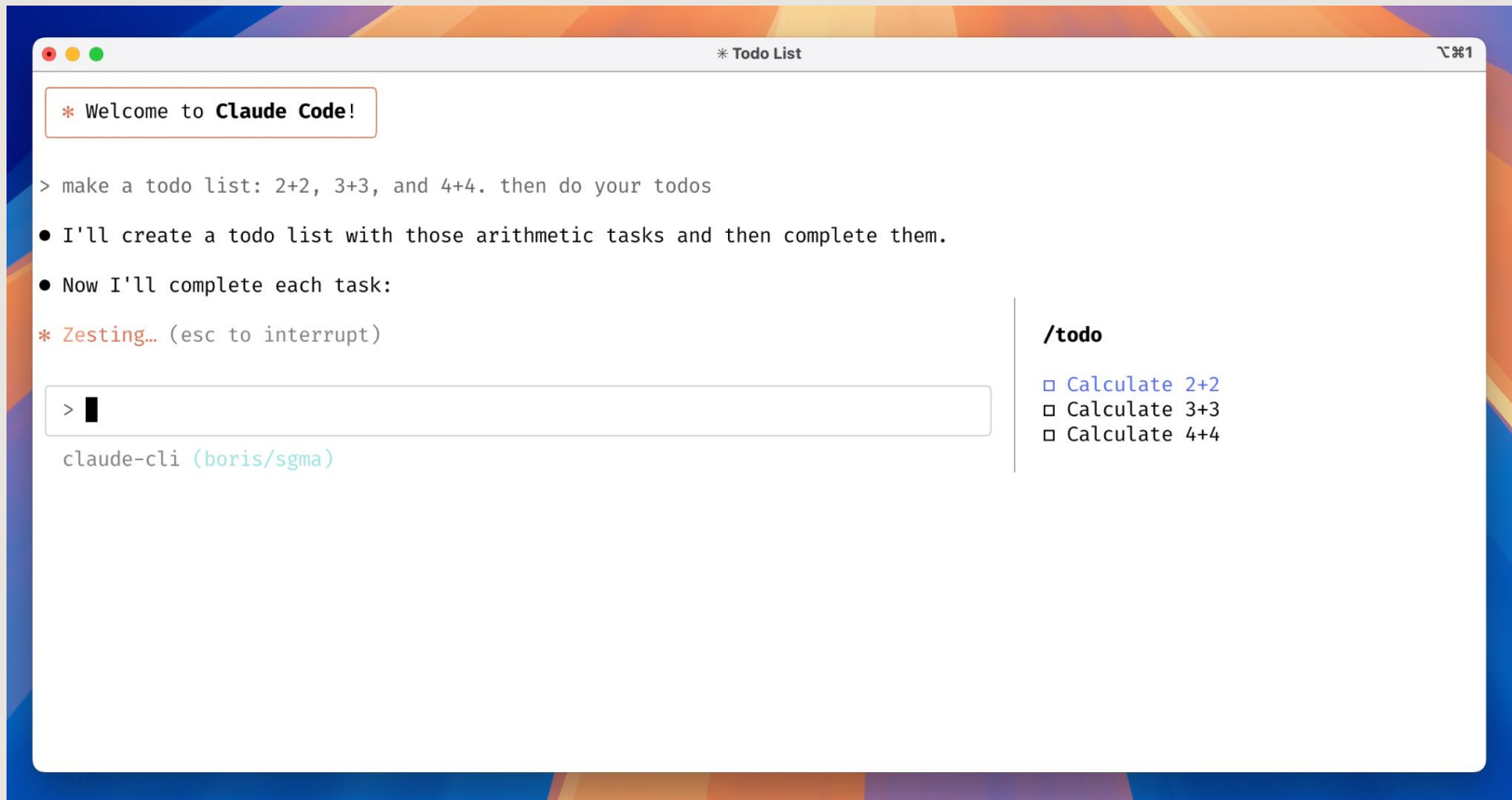
> make it so instead of todos showing up as they come in, we hide the tool use and result for todos, and render a fixed todo list above the input. title it "/todo (1 of 3)" in grey



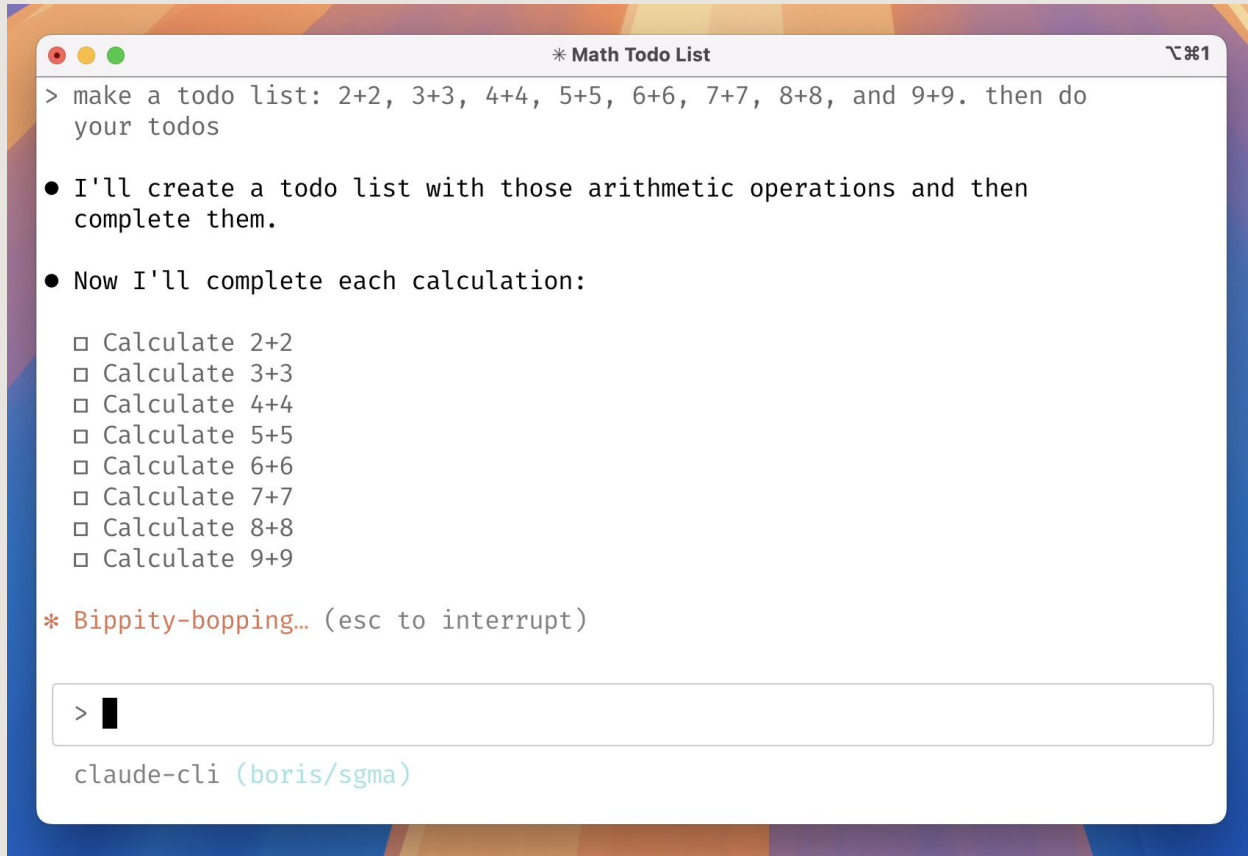
> actually don't show a todo list at all, and instead render the tool uses inline, as bold headings when the model starts working on a todo. keep the "step 2 of 4" or whatever, and add middot /todo to see after in grey



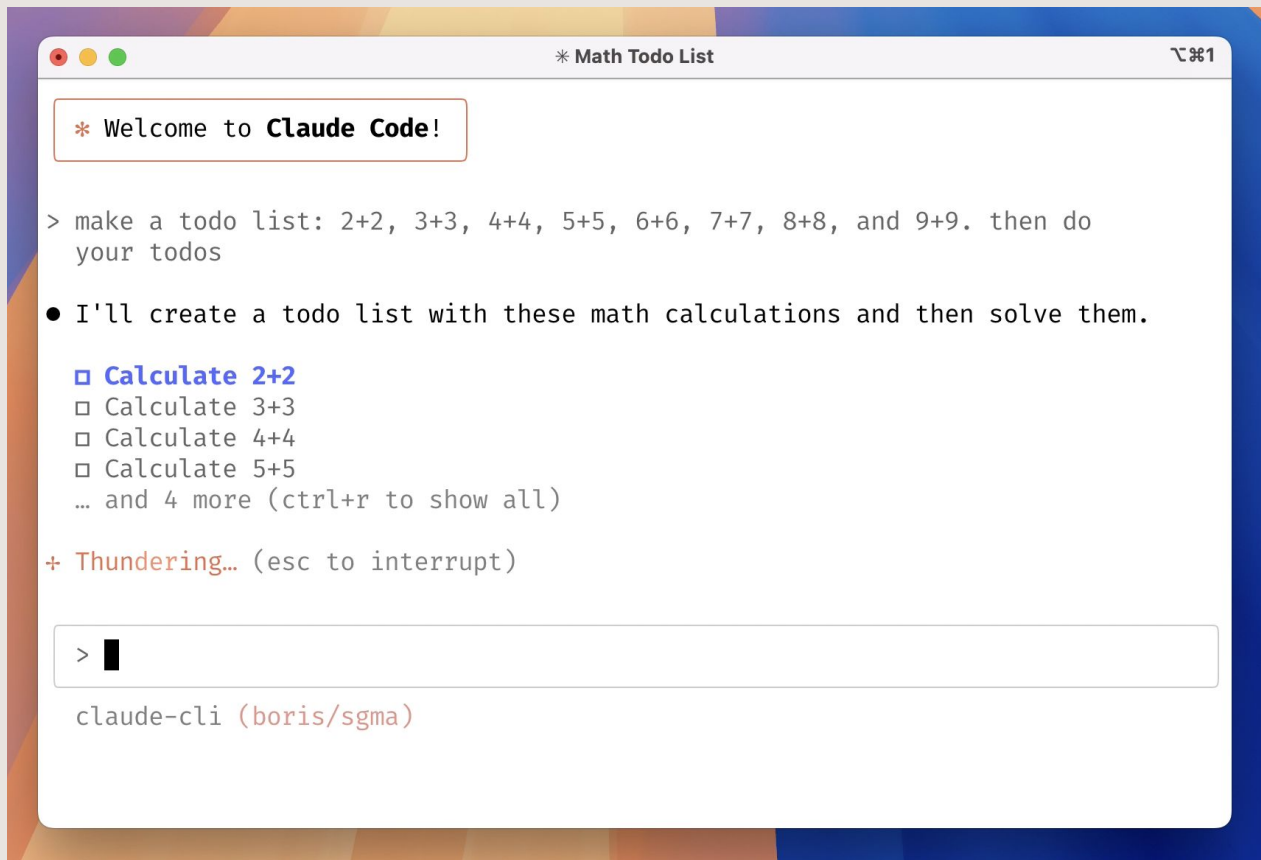
> also add a todo pill under the text input, similar to bg tasks. it should render "todos: 1 of 3" or whatever. make the pill interactive



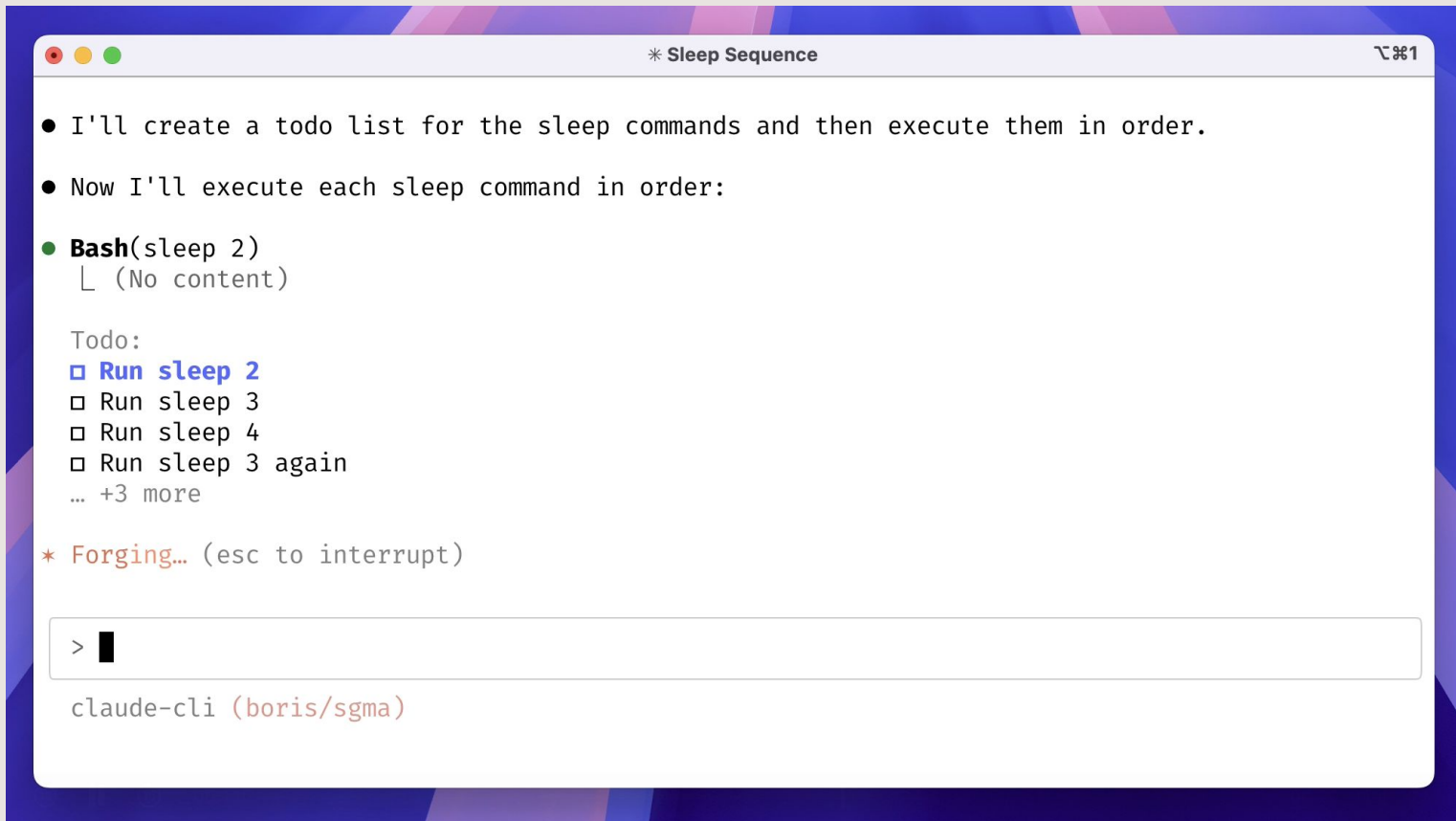
> actually undo both the pill and headings. instead, make the todo list render to the right of the input, vertically centered with a grey divider. show it when todos are active, hide it when it's done



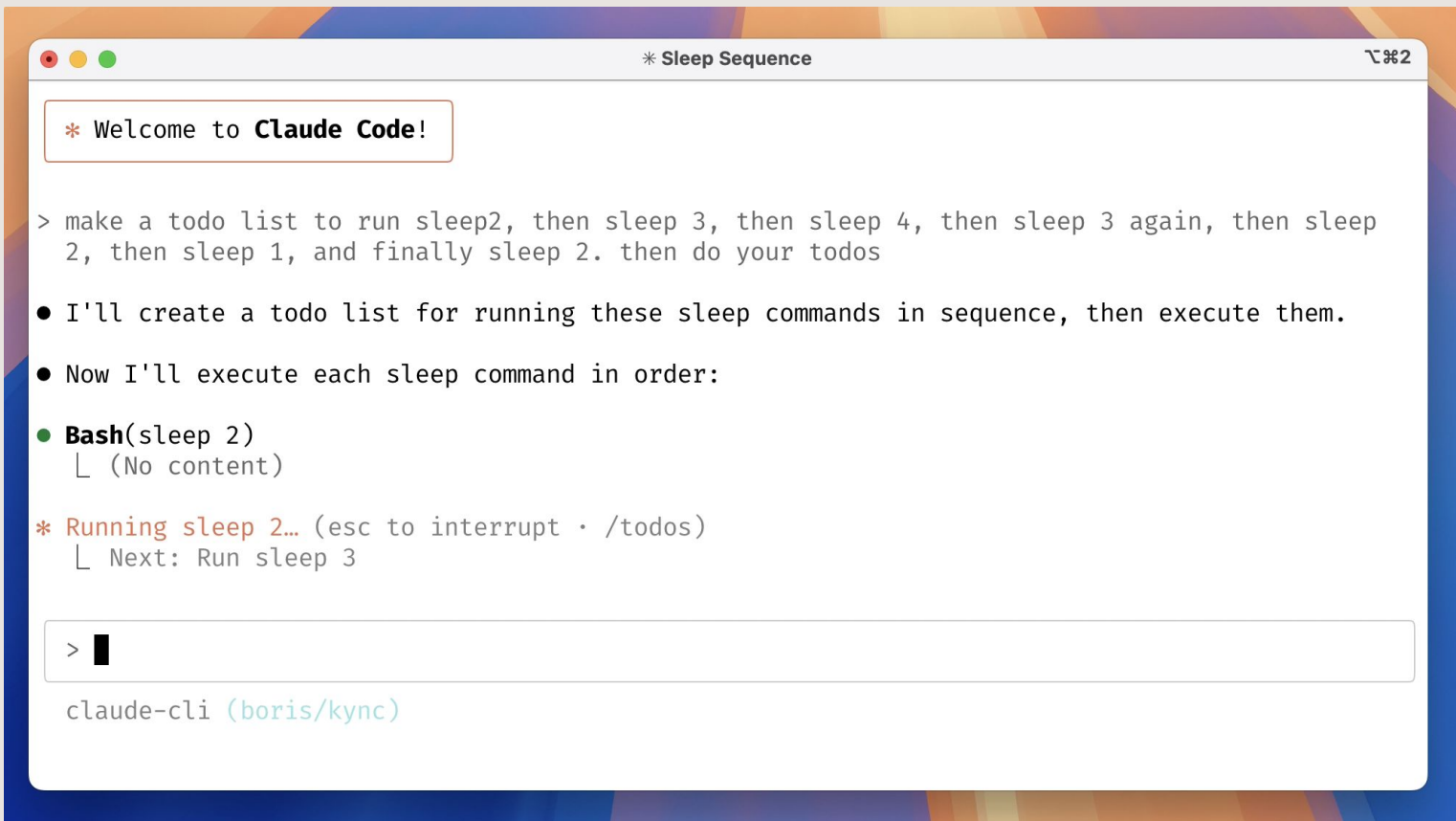
> actually what if you show the todo list above the input instead. truncate at 5 and show "... and 4 more" or whatever. add a heading "Todo:" in grey text



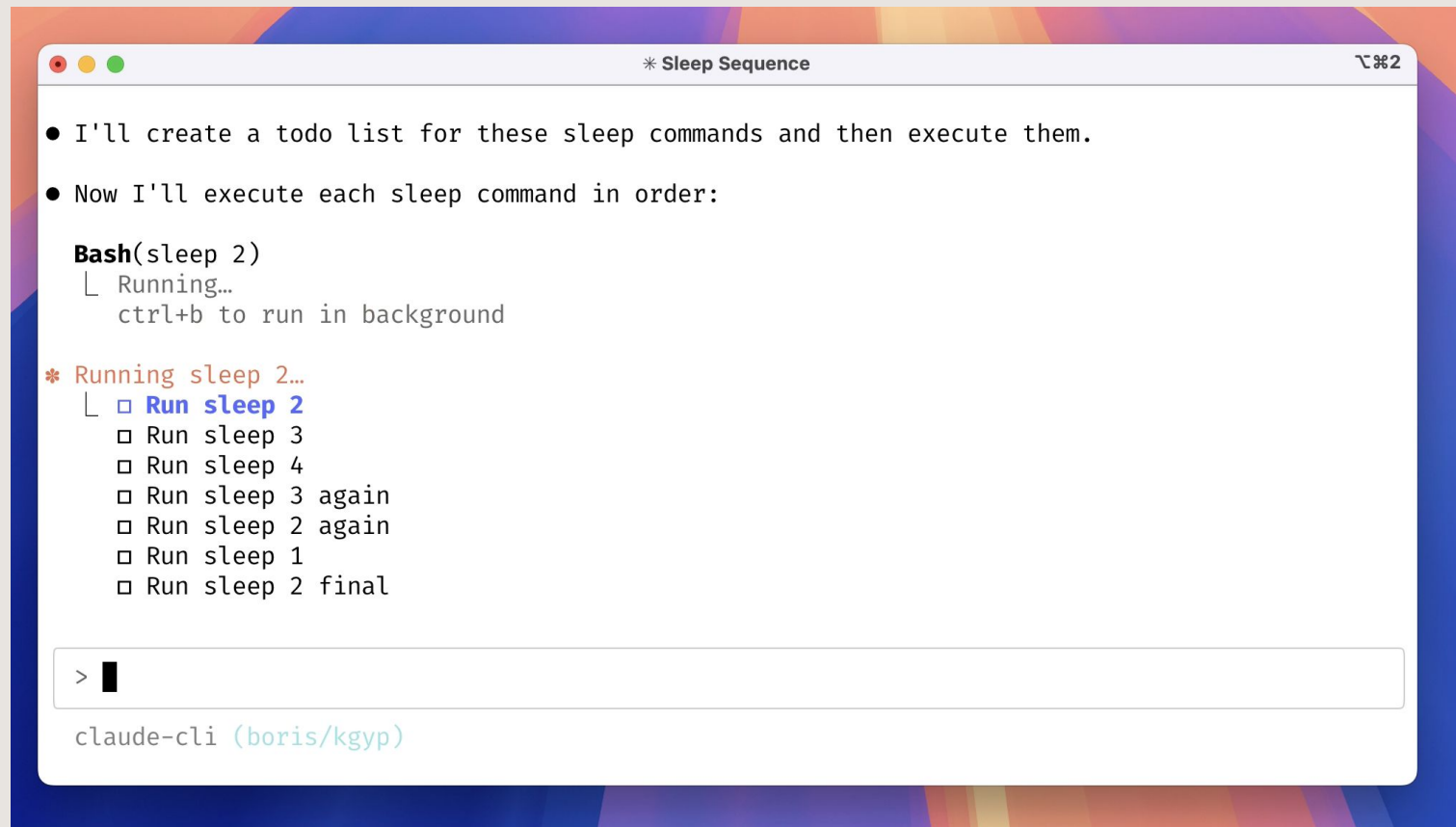
> actually what if you show the todo list above the input instead. truncate at 5 and show "... and 4 more" or whatever. add a heading "Todo:" in grey text



> actually what if you show the todo list above the input instead. truncate at 5 and show "... and 4 more" or whatever. add a heading "Todo:" in grey text



> instead of showing todos above the input, merge them into the spinner. show the current todo as the spinner message in active verb form, and the next todo instead of the spinner tip, in passive form. also add a /todos slash command to see all todos, and mention it next to "esc to interrupt"



> make it so i can hit ctrl+t to expand todos inline. also rm the "esc to interrupt" message when we show the todos, and replace it w "ctrl+t to expand"

* Welcome to **Claude Code**!

> make a todo list 2+2, 3+3, 4+4. dont do it yet

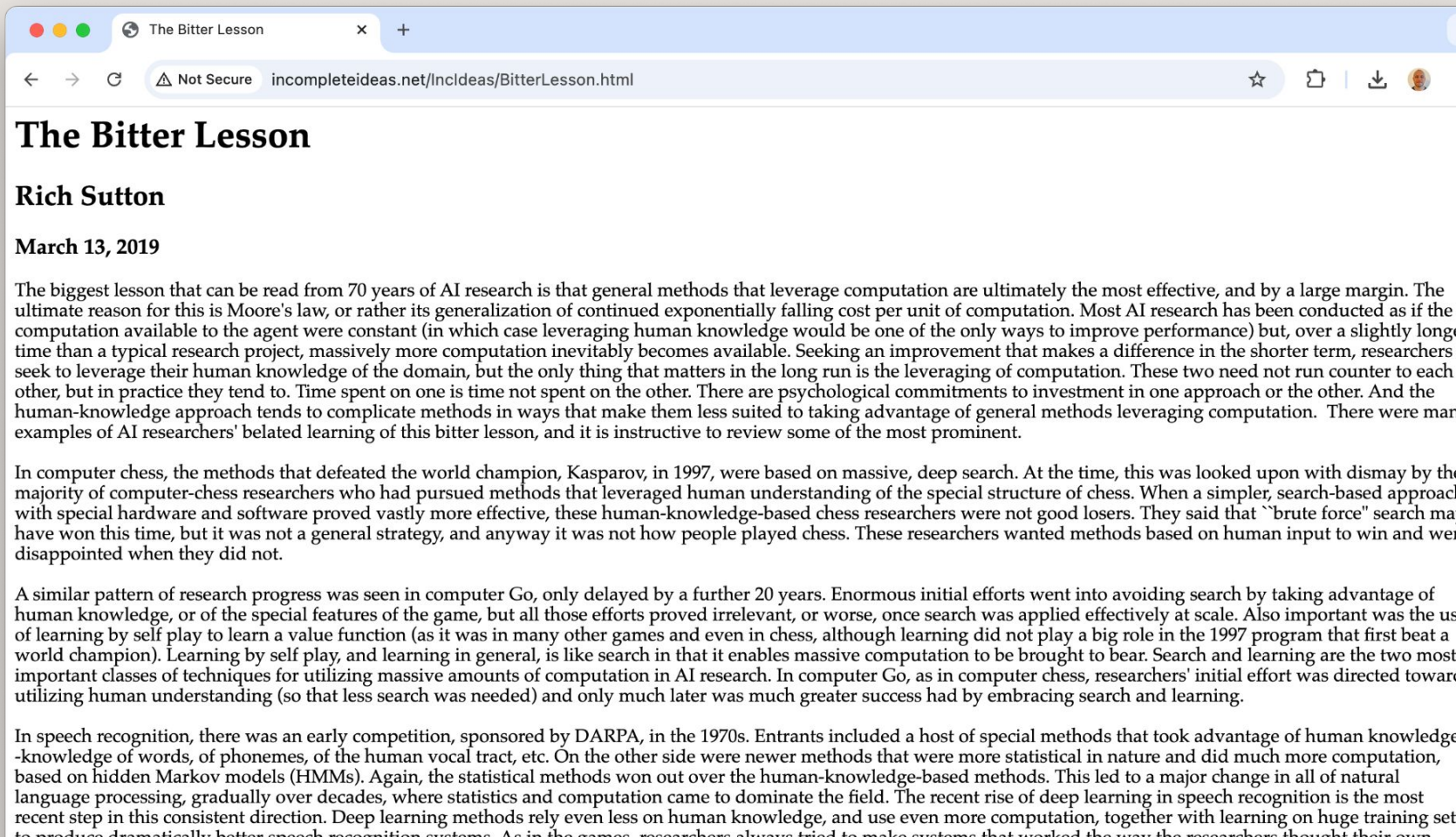
claude-cli (boris/tgez)

1. build for the model six months from now

2. be ready to evolve

3. ask not what the model can do for you

4.



The screenshot shows a web browser window with the title bar "The Bitter Lesson". The address bar shows the URL "incompleteideas.net/InIdeas/BitterLesson.html" with a "Not Secure" warning. The page content includes the title "The Bitter Lesson", the author "Rich Sutton", and the date "March 13, 2019". The main text discusses the importance of leveraging computation in AI research, contrasting it with human knowledge-based methods. It mentions Moore's law and the exponential growth of computation. The text is divided into paragraphs, with the first paragraph being the longest. The browser interface includes standard navigation buttons (back, forward, refresh) and a search bar.

The Bitter Lesson

Rich Sutton

March 13, 2019

The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin. The ultimate reason for this is Moore's law, or rather its generalization of continued exponentially falling cost per unit of computation. Most AI research has been conducted as if the computation available to the agent were constant (in which case leveraging human knowledge would be one of the only ways to improve performance) but, over a slightly longer time than a typical research project, massively more computation inevitably becomes available. Seeking an improvement that makes a difference in the shorter term, researchers seek to leverage their human knowledge of the domain, but the only thing that matters in the long run is the leveraging of computation. These two need not run counter to each other, but in practice they tend to. Time spent on one is time not spent on the other. There are psychological commitments to investment in one approach or the other. And the human-knowledge approach tends to complicate methods in ways that make them less suited to taking advantage of general methods leveraging computation. There were many examples of AI researchers' belated learning of this bitter lesson, and it is instructive to review some of the most prominent.

In computer chess, the methods that defeated the world champion, Kasparov, in 1997, were based on massive, deep search. At the time, this was looked upon with dismay by the majority of computer-chess researchers who had pursued methods that leveraged human understanding of the special structure of chess. When a simpler, search-based approach with special hardware and software proved vastly more effective, these human-knowledge-based chess researchers were not good losers. They said that "brute force" search may have won this time, but it was not a general strategy, and anyway it was not how people played chess. These researchers wanted methods based on human input to win and were disappointed when they did not.

A similar pattern of research progress was seen in computer Go, only delayed by a further 20 years. Enormous initial efforts went into avoiding search by taking advantage of human knowledge, or of the special features of the game, but all those efforts proved irrelevant, or worse, once search was applied effectively at scale. Also important was the use of learning by self play to learn a value function (as it was in many other games and even in chess, although learning did not play a big role in the 1997 program that first beat a world champion). Learning by self play, and learning in general, is like search in that it enables massive computation to be brought to bear. Search and learning are the two most important classes of techniques for utilizing massive amounts of computation in AI research. In computer Go, as in computer chess, researchers' initial effort was directed toward utilizing human understanding (so that less search was needed) and only much later was much greater success had by embracing search and learning.

In speech recognition, there was an early competition, sponsored by DARPA, in the 1970s. Entrants included a host of special methods that took advantage of human knowledge of human knowledge of words, of phonemes, of the human vocal tract, etc. On the other side were newer methods that were more statistical in nature and did much more computation, based on hidden Markov models (HMMs). Again, the statistical methods won out over the human-knowledge-based methods. This led to a major change in all of natural language processing, gradually over decades, where statistics and computation came to dominate the field. The recent rise of deep learning in speech recognition is the most recent step in this consistent direction. Deep learning methods rely even less on human knowledge, and use even more computation, together with learning on huge training sets to produce dramatically better speech recognition systems. As in the games, researchers always tried to make systems that worked the way the researchers thought their own

thanks!

```
npm install -g @anthropic-ai/claude-code
```

```
claude.ai/code
```

```
boris@anthropic.com
```